

TECHNICAL TRAINING COURSE

Overview of Web Services



Comprehensive Course Materials

Topics Covered: Why Web Services • SOA • HTTP & XML • SOAP • WSDL
REST • UDDI • WS-I Basic Profile • Security

Duration: 2 Days (16 hours)

Level: Intermediate – Developer / Architect

Version: 1.0

Table of Contents

Table of Contents	2
1. Why Web Services?	5
1.1 Learning Objectives	5
1.2 The Integration Problem	5
1.3 Limitations of Earlier Approaches	5
1.4 What Web Services Bring	5
1.5 Review Questions	6
2. Service-Oriented Architecture (SOA)	7
2.1 Learning Objectives	7
2.2 What is SOA?	7
2.3 Core SOA Principles	7
2.4 The SOA Reference Architecture	7
2.5 SOA Roles: Provider, Consumer, and Broker	8
2.6 Review Questions	8
3. HTTP and XML	9
3.1 Learning Objectives	9
3.2 HTTP as the Transport	9
3.3 HTTP Methods	9
3.4 Key HTTP Headers in Web Services	9
3.5 HTTP Status Codes	10
3.6 XML Fundamentals	10
3.7 XML Namespaces	10
3.8 XML Schema (XSD)	11
3.9 Review Questions	11
4. SOAP	12
4.1 Learning Objectives	12
4.2 What is SOAP?	12
4.3 SOAP Message Structure	12
4.4 Sample SOAP 1.2 Request	12
4.5 Sample SOAP 1.2 Response	13
4.6 SOAP Fault Structure	13
4.7 SOAP 1.1 vs SOAP 1.2	13
4.8 mustUnderstand Attribute	14
4.9 Review Questions	14
5. WSDL – Web Services Description Language	15
5.1 Learning Objectives	15
5.2 What is WSDL?	15

5.3 WSDL 1.1 Document Structure	15
5.4 WSDL 1.1 Example (abridged)	15
5.5 Operation Exchange Patterns.....	16
5.6 Document vs RPC Style	17
5.7 WSDL 2.0 Key Changes.....	17
5.8 Review Questions	17
6. The SOAP Vision	18
6.1 Learning Objectives	18
6.2 The Original Vision	18
6.3 The WS-* Extension Stack	18
6.4 Where SOAP Still Excels.....	19
6.5 SOAP Message Processing Model.....	19
6.6 Review Questions.....	19
7. The REST Vision	20
7.1 Learning Objectives	20
7.2 Origins of REST	20
7.3 The Six REST Constraints.....	20
7.4 Resources and URIs.....	20
7.5 URI Design Principles.....	21
7.6 HTTP Status Codes in REST	21
7.7 HATEOAS.....	21
7.8 REST vs SOAP Comparison	21
7.9 Richardson Maturity Model	22
7.10 Review Questions	22
8. UDDI – Universal Description, Discovery and Integration	23
8.1 Learning Objectives	23
8.2 What is UDDI?	23
8.3 The Three UDDI Information Structures	23
8.4 UDDI Data Structures	23
8.5 Why Public UDDI Registries Failed	23
8.6 What Replaced UDDI?	24
8.7 Review Questions.....	24
9. The WS-I Basic Profile	25
9.1 Learning Objectives	25
9.2 What is WS-I?	25
9.3 Why was the Basic Profile Needed?.....	25
9.4 Key Basic Profile 1.1 Conformance Requirements.....	25
9.5 Basic Profile 2.0	26
9.6 Testing for Conformance	26

- 9.7 Common Non-Conformance Issues 26
- 9.8 Review Questions 26
- 10. Security 28
 - 10.1 Learning Objectives 28
 - 10.2 Security Threat Landscape 28
 - 10.3 Transport-Level Security (TLS/HTTPS)..... 28
 - 10.4 Authentication Methods 28
 - 10.5 WS-Security for SOAP 29
 - 10.6 OAuth 2.0 for REST APIs 29
 - 10.7 XML-Specific Vulnerabilities 30
 - 10.8 Web Service Security Checklist..... 31
 - 10.9 Review Questions 31
- Appendix A: Quick Reference 32
 - A.1 Key Standards and Specifications..... 32
 - A.2 Key Glossary 32
 - A.3 Course Assessment 33

1. Why Web Services?

1.1 Learning Objectives

- Understand the business and technical drivers behind web services adoption
- Identify the limitations of earlier integration approaches
- Articulate the value proposition of a standards-based integration layer

1.2 The Integration Problem

Modern organisations run dozens—sometimes hundreds—of business systems: ERP, CRM, billing, logistics, HR, and custom line-of-business applications. Making these systems talk to one another is one of the most persistent challenges in enterprise software.

Historically, integration was achieved through:

- Point-to-point custom connections (spaghetti integration)
- Proprietary middleware such as CORBA or DCOM
- EDI over VANs (Value-Added Networks)
- Proprietary vendor connectors that lock organisations to a single stack

The Integration Tax

Gartner estimated that 40–70 % of IT budgets in large enterprises were consumed by integration work—not building new features, but wiring existing systems together. Web services emerged as a way to standardise that wiring.

1.3 Limitations of Earlier Approaches

Approach	Strength	Limitation
Custom code / FTP	Simple point-to-point	Brittle; breaks with any change; $O(n^2)$ complexity
CORBA / IIOP	Object-level interoperability	Complex ORB config; firewall-hostile; vendor lock-in
DCOM / COM+	Native Windows integration	Windows-only; NAT/firewall problems
EDI / X12	Industry standard for documents	Batch-oriented; expensive VAN fees; rigid schema
Proprietary MOM	Reliable messaging	Vendor lock-in; requires same MOM at both ends

1.4 What Web Services Bring

Web services address these limitations by combining three universally available technologies:

1. HTTP – the transport already allowed through every firewall
2. XML – a text-based, platform-neutral data format
3. Open standards (SOAP, WSDL, UDDI) – agreed upon by an industry-wide consortium

Key benefits include:

- Platform independence – Java client calling a .NET service calling a COBOL back-end
- Firewall friendliness – operates over standard port 80/443
- Loose coupling – services expose contracts, not implementation details
- Discoverability – registries allow services to be found at runtime
- Composability – services can be orchestrated into higher-order business processes

Key Principle

Web services shift integration from 'how do I connect to this specific system?' to 'what contract does this service expose?' – enabling true platform-neutral interoperability.

1.5 Review Questions

4. List three specific problems caused by point-to-point integration at enterprise scale.
5. Why is HTTP an advantage as a transport for integration traffic?
6. What does 'loose coupling' mean in the context of service design?

2. Service-Oriented Architecture (SOA)

2.1 Learning Objectives

- Define SOA and its core principles
- Distinguish SOA from object-oriented and component-based approaches
- Understand the SOA reference architecture layers

2.2 What is SOA?

Service-Oriented Architecture (SOA) is an architectural style in which software components expose their functionality as network-accessible services. Each service:

- Has a well-defined interface (contract)
- Is self-contained and stateless where possible
- Communicates through messages, not in-memory calls
- Can be discovered, invoked, and composed independently of technology

SOA is not a product or technology—it is a set of design principles applied at the enterprise level.

2.3 Core SOA Principles

Term	Definition
Standardised Contract	Services expose behaviour via formal interface definitions (WSDL, schemas).
Loose Coupling	Service consumers depend on the contract, not on implementation specifics.
Abstraction	Only the interface is public; internals are hidden from consumers.
Reusability	Services are designed for reuse across multiple applications.
Autonomy	Services control their own logic and runtime environment.
Statelessness	Services avoid retaining state between calls to aid scalability.
Discoverability	Services publish metadata so they can be found by consumers.
Composability	Services can be combined into composite services or workflows.

2.4 The SOA Reference Architecture

A typical SOA reference architecture contains the following layers:

- Consumer Layer – browsers, mobile apps, partner portals, and other services
- Process / Orchestration Layer – BPEL or similar workflows combining multiple services
- Service Layer – business services exposing domain capabilities
- Component Layer – fine-grained components assembled into services
- Operational Systems Layer – databases, ERPs, legacy mainframes

An Enterprise Service Bus (ESB) typically sits between the layers, providing routing, transformation, and mediation.

SOA vs Microservices

SOA typically implies an ESB as a central integration hub with coarse-grained services. Microservices favour decentralised communication, smaller services, and independent deployment.

Both share the principle of service encapsulation but differ in governance and granularity.

2.5 SOA Roles: Provider, Consumer, and Broker

Role	Responsibility	Example
Service Provider	Implements and publishes the service	A bank exposing an account-balance service
Service Consumer	Discovers and invokes the service	A mobile app requesting a balance check
Service Broker	Maintains a registry of available services	UDDI registry, API Gateway catalogue

2.6 Review Questions

7. Explain the difference between 'loose coupling' and 'abstraction' as SOA principles.
8. In what layer of the reference architecture would a BPEL process sit?
9. How does an ESB contribute to the SOA pattern?

3. HTTP and XML

3.1 Learning Objectives

- Understand the HTTP request/response lifecycle relevant to web services
- Identify key HTTP methods, headers, and status codes used in web services
- Understand the structure and role of XML in web service messaging
- Apply XML namespaces correctly in service messages

3.2 HTTP as the Transport

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the World Wide Web. Web services leverage HTTP because it is:

- Universally supported by all platforms and languages
- Already permitted through corporate firewalls on ports 80 (HTTP) and 443 (HTTPS)
- Stateless by design—each request is self-contained
- Supported by mature load-balancers, caches, and proxies

3.3 HTTP Methods

Method	Meaning	Idempotent?	Safe?	Web Service Use
GET	Retrieve a resource	Yes	Yes	REST: read operations; rarely in SOAP
POST	Submit data / create resource	No	No	SOAP: all calls; REST: create/action
PUT	Replace a resource completely	Yes	No	REST: full updates
PATCH	Partial update of a resource	No	No	REST: partial updates
DELETE	Remove a resource	Yes	No	REST: delete operations
HEAD	Retrieve headers only	Yes	Yes	Health checks; metadata
OPTIONS	Describe allowed methods (CORS)	Yes	Yes	Pre-flight requests in AJAX

3.4 Key HTTP Headers in Web Services

Term	Definition
Content-Type	Describes the media type of the body (e.g., application/xml, application/json).
Accept	Tells the server what media type the client wants in the response.
SOAPAction	Required SOAP 1.1 header indicating the intent of the SOAP request.

Authorization	Carries credentials for authentication (Basic, Bearer token, etc.).
Cache-Control	Directs caches on how to handle the response.
Content-Length	The size of the request or response body in bytes.

3.5 HTTP Status Codes

Range	Category	Common Codes & Meanings
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

3.6 XML Fundamentals

XML (eXtensible Markup Language) is the data format underpinning SOAP. A well-formed XML document must:

- Have exactly one root element
- Close every opened tag
- Properly nest all elements
- Use valid attribute syntax
- Escape reserved characters: & < > " '

```
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="http://example.com/orders">
  <orderId>ORD-2024-001</orderId>
  <customer>
    <name>Acme Corporation</name>
    <tier>premium</tier>
  </customer>
  <lineItems>
    <item sku="W-101" qty="5">Widget</item>
  </lineItems>
  <total currency="GBP">499.99</total>
</order>
```

3.7 XML Namespaces

Namespaces prevent name collisions when combining XML vocabularies from different sources. The xmlns attribute associates an element with a URI:

```
<!-- Default namespace applies to all unqualified elements -->
<env:Envelope
  xmlns:env="http://www.w3.org/2003/05/soap-envelope"
  xmlns:ord="http://example.com/orders">
  <env:Header/>
```

```
<env:Body>
  <ord:PlaceOrder>
    <ord:orderId>ORD-2024-001</ord:orderId>
  </ord:PlaceOrder>
</env:Body>
</env:Envelope>
```

Important Note on Namespace URIs

A namespace URI is an identifier, not a URL. The URI does not need to resolve to a real web page—it simply needs to be globally unique. Using your organisation's domain as a prefix (e.g., `http://yourcompany.com/...`) is a common convention.

3.8 XML Schema (XSD)

XML Schema Definition (XSD) documents describe the legal structure of an XML document—what elements and attributes are permitted, their data types, and cardinality constraints. WSDL uses XSD to define the types used in service messages.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="orderId">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:pattern value="ORD-\d{4}-\d{3}"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

3.9 Review Questions

10. Why is HTTP particularly well-suited as the transport for web services?
11. What is the difference between PUT and PATCH?
12. Explain why a namespace URI does not need to resolve to an actual web page.
13. What constraint does XSD add on top of XML well-formedness?

4. SOAP

4.1 Learning Objectives

- Describe the structure of a SOAP message and the role of each element
- Differentiate SOAP 1.1 and SOAP 1.2
- Construct valid SOAP request and response messages
- Understand SOAP fault handling

4.2 What is SOAP?

SOAP (Simple Object Access Protocol) is a W3C standard messaging protocol for exchanging structured information in the implementation of web services. A SOAP message is an XML document with a defined structure that enables:

- Platform-neutral RPC (Remote Procedure Calls)
- Document-style message exchange
- Extensible message headers for cross-cutting concerns
- A standardised error format

4.3 SOAP Message Structure

Every SOAP message is an XML document with three key elements:

Term	Definition
Envelope	The root element; declares the SOAP namespace and defines the message structure.
Header	(Optional) Contains metadata such as authentication tokens, routing info, or WS-* extensions.
Body	Contains the main payload—either the operation being invoked or the response data.
Fault	(In response Body) Carries error information when processing fails.

4.4 Sample SOAP 1.2 Request

```
POST /CustomerService HTTP/1.1
Host: api.example.com
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 412

<?xml version="1.0" encoding="UTF-8"?>
<env:Envelope
  xmlns:env="http://www.w3.org/2003/05/soap-envelope"
  xmlns:cust="http://example.com/customer">
  <env:Header>
    <cust:AuthToken env:mustUnderstand="true">
      eyJhbGciOiJIUzI1NiJ9...
    </cust:AuthToken>
  </env:Header>
```

```
<env:Body>
  <cust:GetCustomerRequest>
    <cust:customerId>CUST-001</cust:customerId>
  </cust:GetCustomerRequest>
</env:Body>
</env:Envelope>
```

4.5 Sample SOAP 1.2 Response

```
HTTP/1.1 200 OK
Content-Type: application/soap+xml; charset=utf-8

<?xml version="1.0" encoding="UTF-8"?>
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope"
  xmlns:cust="http://example.com/customer">
  <env:Body>
    <cust:GetCustomerResponse>
      <cust:customer>
        <cust:id>CUST-001</cust:id>
        <cust:name>Acme Corporation</cust:name>
        <cust:tier>PREMIUM</cust:tier>
        <cust:creditLimit currency="GBP">50000</cust:creditLimit>
      </cust:customer>
    </cust:GetCustomerResponse>
  </env:Body>
</env:Envelope>
```

4.6 SOAP Fault Structure

When a SOAP request cannot be processed, the server returns an HTTP 500 (SOAP 1.1) or 400/500 (SOAP 1.2) with a Fault element in the Body:

```
<env:Body>
  <env:Fault>
    <env:Code>
      <env:Value>env:Sender</env:Value>    <!-- Client error -->
    </env:Code>
    <env:Reason>
      <env:Text xml:lang="en">Customer ID not found: CUST-999</env:Text>
    </env:Reason>
    <env:Detail>
      <err:ErrorCode xmlns:err="http://example.com/errors">ERR-
404</err:ErrorCode>
    </env:Detail>
  </env:Fault>
</env:Body>
```

4.7 SOAP 1.1 vs SOAP 1.2

Feature	SOAP 1.1	SOAP 1.2
Namespace URI	http://schemas.xmlsoap.org/soap/envelope/	http://www.w3.org/2003/05/soap-envelope
Content-Type	text/xml	application/soap+xml

SOAPAction header	Required (can be empty string)	Optional; carried in Content-Type params
Fault codes	Server / Client / VersionMismatch	Receiver / Sender / VersionMismatch / DataEncodingUnknown
W3C status	Note (not a W3C Recommendation)	W3C Recommendation
Binding options	HTTP, SMTP, others	HTTP mandatory; others optional

4.8 mustUnderstand Attribute

A header block may carry the `env:mustUnderstand="true"` attribute. If the receiving node does not understand that header block, it **MUST** reject the message with a fault. This mechanism allows mandatory processing requirements (authentication, routing, transactions) to be enforced by intermediaries.

4.9 Review Questions

14. What are the three mandatory structural elements of every SOAP Envelope?
15. When should you use SOAP 1.2 over SOAP 1.1?
16. What happens when a node receives a mustUnderstand header it does not recognise?
17. Draft a SOAP Fault response for the scenario: the submitted date field is not a valid ISO 8601 date.

5. WSDL – Web Services Description Language

5.1 Learning Objectives

- Understand the purpose and structure of a WSDL document
- Read and interpret WSDL 1.1 and WSDL 2.0 documents
- Map WSDL constructs to the SOAP binding they describe

5.2 What is WSDL?

WSDL (Web Services Description Language) is an XML-based interface description language that describes the functionality offered by a web service. Think of WSDL as the service's formal contract—it tells consumers exactly what operations are available, what messages they exchange, and where to find the service.

Analogy

If a web service is a vending machine, WSDL is the label on the front: it lists every product (operation), what coin combination to insert (request message), and what the machine dispenses (response message). It also shows the machine's location (endpoint address).

5.3 WSDL 1.1 Document Structure

Term	Definition
<types>	XSD schema definitions for all message types used by the service.
<message>	Named parameter lists—each message defines the data exchanged in one direction.
<portType>	Defines the abstract interface—a named collection of operations (abstract contract).
<binding>	Specifies the concrete protocol and data format for a particular portType.
<service>	Maps a binding to one or more network addresses (endpoints).
<port>	A single endpoint: one binding plus one address.

5.4 WSDL 1.1 Example (abridged)

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="CustomerService"
  targetNamespace="http://example.com/customer"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap12/"
  xmlns:tns="http://example.com/customer"
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

```
<!-- 1. Types -->
<types>
  <xs:schema targetNamespace="http://example.com/customer">
    <xs:element name="GetCustomerRequest">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="customerId" type="xs:string"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
    <xs:element name="GetCustomerResponse">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="name" type="xs:string"/>
          <xs:element name="tier" type="xs:string"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:schema>
</types>

<!-- 2. Messages -->
<message name="GetCustomerIn">
  <part name="body" element="tns:GetCustomerRequest"/>
</message>
<message name="GetCustomerOut">
  <part name="body" element="tns:GetCustomerResponse"/>
</message>

<!-- 3. PortType (abstract interface) -->
<portType name="CustomerPortType">
  <operation name="GetCustomer">
    <input message="tns:GetCustomerIn"/>
    <output message="tns:GetCustomerOut"/>
  </operation>
</portType>

<!-- 4. Binding -->
<binding name="CustomerSoap12Binding" type="tns:CustomerPortType">
  <soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http"/>
  <operation name="GetCustomer">
    <soap:operation soapAction="GetCustomer"/>
    <input><soap:body use="literal"/></input>
    <output><soap:body use="literal"/></output>
  </operation>
</binding>

<!-- 5. Service -->
<service name="CustomerService">
  <port name="CustomerPort" binding="tns:CustomerSoap12Binding">
    <soap:address location="https://api.example.com/CustomerService"/>
  </port>
</service>

</definitions>
```

5.5 Operation Exchange Patterns

Pattern	Elements	Description
---------	----------	-------------

Request-Response	<input> + <output>	Client sends; service replies. Most common.
One-Way	<input> only	Client sends; no reply expected (fire-and-forget).
Solicit-Response	<output> + <input>	Service initiates; client replies.
Notification	<output> only	Service sends; no client reply expected.

5.6 Document vs RPC Style

Aspect	Document / Literal	RPC / Encoded
Body Content	Arbitrary XML element defined in XSD	Operation wrapper element + typed parts
Schema Validation	Fully validatable against XSD	Cannot validate body standalone
WS-I Compliance	Compliant (document/literal wrapped)	RPC/encoded is non-compliant
Recommended?	Yes — preferred for new services	Avoid for interoperable services

Best Practice

Always use document/literal/wrapped style for new SOAP services.
This is the only style that is both WS-I compliant and cleanly schema-validatable.

5.7 WSDL 2.0 Key Changes

- portType renamed to interface
- message elements removed; types referenced directly in operation
- Cleaner HTTP binding with explicit support for GET/POST
- Fault definitions improved with separate fault element declarations
- Less verbose; better aligned with REST-style services

5.8 Review Questions

18. What is the difference between a portType and a binding in WSDL 1.1?
19. Why is RPC/encoded style discouraged for new web services?
20. Where in a WSDL document would you find the URL to call the service?

6. The SOAP Vision

6.1 Learning Objectives

- Understand the original vision and ecosystem of SOAP-based web services
- Describe the WS-* protocol stack and key extension standards
- Evaluate where SOAP-based services are still the right choice

6.2 The Original Vision

When SOAP was introduced in 1999 (originally authored by Dave Winer, Don Box, Bob Atkinson, and Mohsen Al-Ghosein at Microsoft), the vision was ambitious: a universal integration fabric where any system could call any service over the internet, using a common XML messaging format, without pre-arranged point-to-point connections.

The vision had three layers:

21. Wire – HTTP + XML (SOAP): universal transport and message format
22. Description – WSDL: machine-readable contracts
23. Discovery – UDDI: runtime service registries

Together these were called the 'Web Services stack' and were championed by Microsoft, IBM, BEA, and Sun.

6.3 The WS-* Extension Stack

Because SOAP itself is intentionally minimal, a family of extension specifications—collectively called WS-*—was developed to address enterprise concerns:

Term	Definition
WS-Security	Message-level authentication, signing, and encryption (X.509, SAML tokens, etc.).
WS-ReliableMsg	Guaranteed message delivery with acknowledgement and ordering.
WS-Addressing	Endpoint references and message addressing for asynchronous routing.
WS-AtomicTrans	Distributed two-phase commit transactions across services.
WS-BusinessActivity	Long-running compensatable transactions spanning multiple services.
WS-Policy	Machine-readable policies describing service capabilities and requirements.
WS-Federation	Identity federation across security domains.
WS-Trust	Requesting and issuing security tokens (STS – Security Token Service).
WS-MetadataExch	Runtime retrieval of service metadata (WSDL, policy).

WS-Coordination

Context propagation for distributed activities.

6.4 Where SOAP Still Excels

Despite REST's dominance in consumer APIs, SOAP remains the preferred choice for:

- Enterprise B2B integration requiring contractual SLAs
- Financial services: SWIFT messaging, payment processing
- Healthcare interoperability: HL7 and insurance claim processing
- Scenarios requiring WS-Security with digital signatures and XML encryption
- Formal ACID or compensating transactions via WS-AtomicTransaction / WS-BusinessActivity
- Legacy enterprise systems (SAP, Oracle, IBM mainframes) with existing SOAP interfaces

Practical Guidance

For greenfield APIs or mobile/web client integration: REST or GraphQL is usually the right choice.

For regulated B2B integration, financial messaging, or when contractual guarantees are mandatory:

SOAP + WS-* provides a proven, auditable, standards-based foundation.

6.5 SOAP Message Processing Model

SOAP defines a processing model based on nodes and roles:

- Ultimate Receiver – the final destination that processes the Body
- Intermediary – a SOAP node that processes Header blocks en route (logging, routing, auth)
- Role / Actor – a URI identifying which node is responsible for a Header block

The env:role attribute (SOAP 1.2) or env:actor (SOAP 1.1) on a Header block declares which node must process it. Combined with mustUnderstand, this enables a reliable processing chain.

6.6 Review Questions

24. Name four WS-* specifications and state the enterprise concern each addresses.
25. What is the difference between an Intermediary node and the Ultimate Receiver?
26. In what types of scenarios would you choose SOAP over REST today?

7. The REST Vision

7.1 Learning Objectives

- Understand the architectural constraints that define REST
- Design RESTful resource models, URLs, and representations
- Describe HTTP-native features used by REST services
- Compare REST and SOAP on key architectural dimensions

7.2 Origins of REST

REST (Representational State Transfer) was defined by Roy Fielding in his 2000 doctoral dissertation at UC Irvine. Fielding was one of the principal authors of the HTTP/1.1 specification, and REST is essentially a formal description of the web's own architecture—using the web's native mechanisms rather than adding a protocol layer on top.

7.3 The Six REST Constraints

Term	Definition
Client-Server	Separation of concerns: UI from data storage; each evolves independently.
Stateless	Each request contains all information needed; no session state stored server-side.
Cacheable	Responses must declare themselves cacheable or not to improve efficiency.
Uniform Interface	A consistent interface between components (resources, representations, self-describing messages, HATEOAS).
Layered System	Client cannot tell whether it is connected directly or through intermediaries.
Code on Demand (opt.)	Servers can extend client functionality by transferring executable code (e.g., JavaScript).

7.4 Resources and URIs

In REST, everything is a resource. A resource is any concept that can be named, and resources are identified by URIs:

```
# Collection resource
GET  /customers          → list all customers
POST /customers          → create a new customer

# Item resource
GET   /customers/{id}    → retrieve customer
PUT   /customers/{id}    → replace customer
PATCH /customers/{id}   → partial update
DELETE /customers/{id}   → delete customer
```

```
# Sub-resource / relationship
GET    /customers/{id}/orders    → orders for a customer
GET    /customers/{id}/orders/{oid} → specific order
```

7.5 URI Design Principles

- Use nouns, not verbs: /customers not /getCustomers
- Use plural nouns for collections: /orders not /order
- Use hyphens, not underscores: /line-items not /line_items
- Keep URIs lowercase
- Do not use file extensions in URIs; use Accept header for format negotiation
- Hierarchical structure should reflect resource relationships

7.6 HTTP Status Codes in REST

Operation	Success Code	Error Codes
GET (found)	200 OK	404 Not Found, 401, 403
POST (create)	201 Created	400 Bad Request, 409 Conflict
PUT / PATCH	200 OK	400, 404, 409, 412 Precondition Failed
DELETE	204 No Content	404, 403
Async operation	202 Accepted	N/A (status polled separately)

7.7 HATEOAS

HATEOAS (Hypermedia as the Engine of Application State) is the REST constraint that makes APIs truly self-describing. Responses include links to related actions, so clients do not need out-of-band knowledge of the API structure:

```
{
  "customerId": "CUST-001",
  "name": "Acme Corporation",
  "tier": "PREMIUM",
  "_links": {
    "self": { "href": "/customers/CUST-001" },
    "orders": { "href": "/customers/CUST-001/orders" },
    "update": { "href": "/customers/CUST-001", "method": "PUT" },
    "delete": { "href": "/customers/CUST-001", "method": "DELETE" }
  }
}
```

7.8 REST vs SOAP Comparison

Dimension	REST	SOAP
Protocol	HTTP native	HTTP (or SMTP, JMS, etc.)

Message format	JSON, XML, plain text, binary	XML only
Interface def.	OpenAPI / Swagger (de facto)	WSDL (formal standard)
Transactions	No built-in; use idempotent design	WS-AtomicTransaction
Security	TLS; OAuth 2.0; JWT	WS-Security; XML encryption
Error handling	HTTP status codes + body	SOAP Fault element
Caching	Native HTTP caching	No caching (POST-only)
Tooling overhead	Low	Higher (code gen, stubs)
Best for	Public APIs, mobile, microservices	Enterprise B2B, regulated industries

7.9 Richardson Maturity Model

The Richardson Maturity Model describes the progression towards a fully RESTful service:

Term	Definition
Level 0 – The Swamp of POX	Single endpoint, single HTTP method (POST), XML payloads (plain old XML).
Level 1 – Resources	Multiple URIs, each representing a distinct resource.
Level 2 – HTTP Verbs	Correct use of HTTP methods and status codes.
Level 3 – Hypermedia	Responses include links (HATEOAS). The 'Glory of REST'.

7.10 Review Questions

27. Name and explain the six REST architectural constraints.
28. Design a URI scheme for a library system with books, authors, and loans.
29. What status code should a POST that creates a resource return, and why?
30. At which Richardson Maturity Level would you classify a typical public API that uses correct verbs and status codes but no hypermedia links?

8. UDDI – Universal Description, Discovery and Integration

8.1 Learning Objectives

- Explain the purpose and structure of a UDDI registry
- Describe the three UDDI information structures
- Assess why public UDDI registries failed and what replaced them

8.2 What is UDDI?

UDDI (Universal Description, Discovery and Integration) is a platform-independent, XML-based registry for businesses worldwide to list their web services. Originally envisioned as a global 'yellow pages' for web services, UDDI was intended to complete the web services triangle alongside SOAP and WSDL.

The web services triangle:

- Publish – providers register their services in UDDI
- Find – consumers search UDDI to discover available services
- Bind – consumers retrieve the WSDL and invoke the service

8.3 The Three UDDI Information Structures

Term	Definition
White Pages	Basic contact information about a business: name, address, phone, email. Analogous to a telephone directory.
Yellow Pages	Businesses categorised by industry, product, and geography using standard taxonomies (NAICS, UNSPSC, ISO 3166).
Green Pages	Technical service information: binding templates, WSDL pointers, endpoint URLs, and access points.

8.4 UDDI Data Structures

Structure	Contains	Analogy
businessEntity	White + Yellow pages data	Company record
businessService	Logical grouping of related services	Department listing
bindingTemplate	Technical endpoint and binding info	Individual phone extension
tModel	Technical model / taxonomy definition	Standard technical specification

8.5 Why Public UDDI Registries Failed

The public UDDI registries (operated by Microsoft, IBM, SAP, and HP) were shut down in 2006. Key reasons:

- Low quality data – no governance or verification of listings
- Wrong granularity – registries worked at company level, not service level
- Runtime discovery not used – services were integrated at design time; nobody queried UDDI at runtime
- Security model – exposing internal service endpoints publicly was unacceptable
- Complexity – the UDDI API was cumbersome to use

8.6 What Replaced UDDI?

Enterprise UDDI lives on as private internal registries, and is largely replaced in practice by:

- API Management Platforms – Apigee, AWS API Gateway, Azure API Management, Kong
- Service Registries – Consul, Eureka (Netflix), etcd (Kubernetes)
- API Catalogues – Swagger Hub, Backstage (Spotify), internal developer portals
- DNS-SD and mDNS – for local service discovery on the network layer

Modern Relevance

The concept behind UDDI—a registry where services are published, described, and discovered—

is more relevant than ever. Every organisation operating microservices maintains an internal service catalogue. UDDI's failure was in execution and governance, not in concept.

8.7 Review Questions

31. Describe the three types of information stored in a UDDI registry.
32. What is a tModel, and why is it important?
33. Give two modern alternatives to UDDI for service discovery.

9. The WS-I Basic Profile

9.1 Learning Objectives

- Understand the purpose and scope of the WS-I Basic Profile
- Identify the key conformance requirements of BP 1.1 and BP 2.0
- Apply Basic Profile rules when designing or reviewing SOAP services

9.2 What is WS-I?

The Web Services Interoperability Organisation (WS-I) was an industry consortium founded in 2002 by Microsoft, IBM, Oracle, BEA, and others. Its mandate was to promote interoperability across platforms, operating systems, and programming languages by publishing profiles that constrain and clarify the underlying standards.

WS-I was absorbed by OASIS in 2010. Its most important publication was the Basic Profile.

9.3 Why was the Basic Profile Needed?

The underlying standards (SOAP, WSDL, XML Schema) were intentionally flexible—too flexible for guaranteed interoperability. Common problems included:

- Services using SOAP 1.1 with RPC/encoded style (not cleanly validatable)
- Inconsistent handling of the SOAPAction HTTP header
- Schema type ambiguities causing parsing failures across toolkits
- Conflicting interpretations of WSDL binding styles

The Basic Profile identifies which subsets and usage patterns of these standards lead to predictably interoperable services.

9.4 Key Basic Profile 1.1 Conformance Requirements

Term	Definition
R1000 – SOAP Version	Services MUST use SOAP 1.1.
R2706 – Binding Style	Services MUST use document/literal style; RPC/encoded is non-conformant.
R2716 – Wrapped Pattern	Document/literal operations SHOULD use the wrapped convention (single element wrapping all parts).
R2744 – SOAPAction Header	The SOAPAction HTTP header MUST be present and match the operation.
R2705 – Schema Usage	XSD schemas embedded in WSDL MUST be valid XML Schema definitions.

R2209 – Fault Messages	Fault elements MUST use the SOAP Fault structure with faultcode and faultstring.
R3003 – HTTP Binding	SOAP messages MUST be sent using HTTP 1.1 or higher.
R1031 – Content-Type	SOAP 1.1 messages MUST use Content-Type: text/xml.

9.5 Basic Profile 2.0

Basic Profile 2.0 updated the constraints to cover SOAP 1.2 and WSDL 1.1 with SOAP 1.2 bindings:

- Services MUST use SOAP 1.2 (namespace <http://www.w3.org/2003/05/soap-envelope>)
- Content-Type MUST be application/soap+xml
- SOAPAction carried in the Content-Type media type parameter, not as a separate header
- Fault code values updated to env:Sender and env:Receiver

9.6 Testing for Conformance

The WS-I provided the Interoperability Testing Tools (ITT), including:

- Analyzer – validates a WSDL and captured message log against BP rules
- Monitor – HTTP proxy that captures message traffic for analysis
- Message Logger – records SOAP traffic with timestamps

Practical Tip

Modern frameworks (Apache CXF, .NET WCF, JAX-WS) generate BP-conformant services by default when using document/literal/wrapped configuration. Always verify generated WSDL with an analyzer tool before exposing a service to external consumers.

9.7 Common Non-Conformance Issues

Issue	Root Cause	Fix
RPC/encoded style	Toolkit default or legacy config	Configure document/literal/wrapped in framework
Missing SOAPAction header	Framework omitting mandatory header	Ensure soapAction attribute set in WSDL binding
Schema type soapenc:Array	SOAP encoding type in types section	Replace with XSD array types (xs:sequence)
Multiple parts in body	Non-wrapped document style	Wrap parts in a single element per BP R2710

9.8 Review Questions

34. What binding style does the WS-I Basic Profile mandate, and why?
35. What is the 'wrapped' document/literal pattern and what problem does it solve?

36. List two tools provided by WS-I for conformance testing.

10. Security

10.1 Learning Objectives

- Identify the key security threats to web services
- Apply transport-level security (TLS/HTTPS) to web service communication
- Describe message-level security using WS-Security
- Implement authentication using API keys, Basic Auth, and OAuth 2.0
- Understand XML-specific security vulnerabilities and mitigations

10.2 Security Threat Landscape

Web services are exposed over the network and are therefore vulnerable to a wide range of attacks. Key threat categories include:

Term	Definition
Eavesdropping	Intercepting messages in transit to read sensitive data.
Tampering	Modifying messages in transit between client and service.
Replay Attack	Capturing a valid message and re-sending it to trigger duplicate actions.
SQL / XML Injection	Injecting malicious content into message data fields to manipulate back-end processing.
DoS / DDoS	Overwhelming the service with requests to deny legitimate access.
Identity Spoofing	Impersonating a legitimate consumer to gain unauthorised access.
XML Bomb	Exponentially expanding XML that exhausts server memory (Billion Laughs attack).
Schema Poisoning	Replacing a schema document with a malicious one to bypass validation.

10.3 Transport-Level Security (TLS/HTTPS)

TLS (Transport Layer Security) is the first line of defence for any web service. It provides:

- Confidentiality – encryption of all traffic between client and server
- Integrity – ensures messages are not tampered with in transit
- Authentication – server (and optionally client) certificates verify identity

Minimum TLS Requirements (2024+)

TLS 1.2 minimum; TLS 1.3 preferred.
Disable SSLv2, SSLv3, TLS 1.0, and TLS 1.1 — all are deprecated and vulnerable.
Use strong cipher suites: ECDHE key exchange, AES-256-GCM or ChaCha20-Poly1305.
Enforce HSTS (HTTP Strict Transport Security) with a minimum max-age of 1 year.
Obtain certificates from a trusted CA; automate renewal (e.g., Let's Encrypt / ACME).

10.4 Authentication Methods

Method	How it Works	Use Case	Risk Level
API Key	Pre-shared secret in header or query param	Machine-to-machine internal APIs	Medium (if over HTTPS)
Basic Auth	Base64-encoded username:password in header	Simple services; never without TLS	High without HTTPS
Digest Auth	MD5 hash of credentials + server nonce	Legacy browser authentication	Medium; use HTTPS anyway
OAuth 2.0	Token-based delegation via authorisation server	User-facing APIs; B2B federation	Low (properly implemented)
JWT (Bearer)	Signed/encrypted token with claims	Stateless auth in REST/SOAP	Low with proper validation
Mutual TLS	Both parties present X.509 certificates	High-assurance B2B connections	Very Low
SAML 2.0	XML-based identity assertion from IdP	Enterprise SSO; WS-Federation	Low (with proper config)

10.5 WS-Security for SOAP

WS-Security (OASIS WSS) provides message-level security for SOAP. Unlike TLS, WS-Security protects the message itself, meaning security persists even when the message passes through intermediaries:

- UsernameToken – embeds credentials directly in the SOAP Header
- X509Token – digital signatures using X.509 certificates
- SAMLToken – SAML assertions for federated identity
- XML Digital Signature – signs specific elements to guarantee integrity
- XML Encryption – encrypts specific elements for selective confidentiality

```
<env:Header>
  <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd"
    env:mustUnderstand="true">
    <wsu:Timestamp xmlns:wsu="..." wsu:Id="TS-001">
      <wsu:Created>2024-01-15T10:00:00Z</wsu:Created>
      <wsu:Expires>2024-01-15T10:05:00Z</wsu:Expires>
    </wsu:Timestamp>
    <wsse:UsernameToken wsu:Id="UT-001">
      <wsse:Username>serviceAccount</wsse:Username>
      <wsse:Password Type="...#PasswordDigest">
        hashedPasswordValue==
      </wsse:Password>
      <wsse:Nonce>randomNonceValue==</wsse:Nonce>
      <wsu:Created>2024-01-15T10:00:00Z</wsu:Created>
    </wsse:UsernameToken>
  </wsse:Security>
</env:Header>
```

10.6 OAuth 2.0 for REST APIs

OAuth 2.0 is the industry standard for delegated authorisation. The key grant types are:

Term	Definition
Client Credentials	Machine-to-machine: client authenticates with client_id + client_secret, receives access token. Use for service-to-service REST calls.
Authorization Code	User-facing: user authenticates at the authorisation server; code exchanged for token. Use when a user must authorise the client.
Auth Code + PKCE	Authorization Code flow with Proof Key for Code Exchange. Replaces implicit grant; required for public clients (SPAs, mobile apps).
Refresh Token	Long-lived token used to obtain new access tokens without re-authentication.

```
# Client Credentials flow - machine-to-machine
POST /oauth/token HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=my-service-client
&client_secret=s3cr3t
&scope=customer:read order:write

# Response
{
  "access_token": "eyJhbGciOiJSUzI1NiJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "customer:read order:write"
}
```

10.7 XML-Specific Vulnerabilities

Vulnerability	Description	Mitigation
XML Bomb / Billion Laughs	Recursive entity expansion exhausts memory	Disable external entity processing; limit entity depth
XXE Injection	External entity loads files or internal URLs	Disable DOCTYPE/DTD processing in XML parser
XPath Injection	Malicious XPath in user input alters query logic	Parameterise XPath expressions; input validation
SOAP Array Attack	Extremely large array declarations crash parsers	Enforce schema validation; limit payload size
Schema Poisoning	Replacing referenced schema with malicious version	Cache schemas locally; verify schema integrity
Coercive Parsing	Deeply nested XML elements exhaust stack space	Limit nesting depth; set parser resource limits

Critical: Disable External Entity Processing

The most dangerous XML vulnerability is XXE (XML External Entity). In Java:

```
DocumentBuilderFactory.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true)
```

```
DocumentBuilderFactory.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
```

Always set these properties on XML parsers; do not rely on framework defaults.

10.8 Web Service Security Checklist

- **Enforce HTTPS/TLS 1.2+ on all endpoints**
- Authenticate every request – no unauthenticated endpoints in production
- Authorise at the resource level (not just the network level)
- Validate all input against published schema before processing
- Set XML parser security features to prevent XXE and entity expansion attacks
- Use timestamps and nonces in WS-Security tokens to prevent replay attacks
- Log all authentication events and anomalies to a SIEM
- Implement rate limiting and throttling on all service endpoints
- Regularly rotate API keys, service account passwords, and certificates
- Scan WSDL and OpenAPI definitions for sensitive data exposure (internal hostnames, etc.)

10.9 Review Questions

37. What is the difference between transport-level security and message-level security? When would you need both?
38. Describe the OAuth 2.0 Client Credentials flow and identify an appropriate use case.
39. Explain the XML Bomb (Billion Laughs) attack. What property must you set on Java's DocumentBuilderFactory to prevent it?
40. A SOAP service passes through a logging intermediary before reaching the ultimate receiver. What security mechanism ensures the message has not been tampered with by the intermediary?

Appendix A: Quick Reference

A.1 Key Standards and Specifications

Term	Definition
SOAP 1.2	W3C Recommendation – http://www.w3.org/TR/soap12/
WSDL 1.1	W3C Note – http://www.w3.org/TR/wsdl
WSDL 2.0	W3C Recommendation – http://www.w3.org/TR/wsdl20/
WS-Security 1.1	OASIS Standard – http://docs.oasis-open.org/wss/
WS-I Basic Profile	OASIS – http://www.ws-i.org/profiles/basicprofile-1.1.html
OAuth 2.0	IETF RFC 6749 – http://tools.ietf.org/html/rfc6749
OpenAPI 3.1	https://spec.openapis.org/oas/v3.1.0
XML Schema	W3C – http://www.w3.org/TR/xmlschema-0/

A.2 Key Glossary

Term	Definition
API	Application Programming Interface – a defined contract for software interactions.
ESB	Enterprise Service Bus – middleware providing routing, transformation, and mediation.
HATEOAS	Hypermedia as the Engine of Application State – REST constraint using links in responses.
Idempotent	An operation whose effect is the same whether executed once or multiple times.
JWT	JSON Web Token – a compact, URL-safe token format for carrying claims.
MTOM	Message Transmission Optimization Mechanism – efficient SOAP binary attachment encoding.
OAuth	Open Authorization – a protocol for delegated access without sharing credentials.
OASIS	Organization for the Advancement of Structured Information Standards.
REST	Representational State Transfer – an architectural style for distributed hypermedia systems.
SAML	Security Assertion Markup Language – XML-based standard for exchanging authentication data.
SOAP	Simple Object Access Protocol – a W3C XML messaging protocol.
STS	Security Token Service – a service that issues security tokens (WS-Trust).
TLS	Transport Layer Security – cryptographic protocol for secure network communication.
UDDI	Universal Description, Discovery and Integration – a web service registry standard.

WSDL	Web Services Description Language – XML format for describing web service interfaces.
WS-*	Family of web service extension specifications (WS-Security, WS-ReliableMessaging, etc.).
XSD	XML Schema Definition – a language for defining the structure and types in XML documents.
XXE	XML External Entity – a class of XML parser vulnerability enabling data exfiltration.

A.3 Course Assessment

Participants are assessed through:

- Lab Exercises – hands-on tasks after each section (50%)
- End-of-course Quiz – 30 multiple-choice and short-answer questions (30%)
- Design Exercise – design a web service interface for a given business scenario (20%)

A pass mark of 70% is required for course certification.